Roteiro de fala

Sistema de Moeda Estudantil · Apresentação final

Tempo total: ~18 min 30 s + perguntas

Antes da aula:

• Apagar a pasta data/ e subir o sistema (saldo zerado para a demo).

• Rodar ./mvnw test e conferir que tudo passa.

• Senha de todas as contas: 123456 (carla.mendes, ana.souza, cantina, admin).

SLIDE 01 de 25

Sistema de Moeda Estudantil

0:00 → 0:30

Boa tarde, professora e pessoal. Eu sou o Gustavo e vou apresentar o Sistema de Moeda Estudantil, do Laboratório 3. A ideia é transformar o reconhecimento do professor em uma moeda virtual, que o aluno troca por vantagens em empresas parceiras. Vou mostrar a arquitetura, a camada de persistência e um tutorial das tecnologias e, no final, o sistema funcionando.

SLIDE 02 de 25

O que vamos ver

0:30 → 1:00

O roteiro tem seis partes. Primeiro, o que o sistema faz e quais requisitos atende. Depois, a arquitetura em camadas, com a segurança e os dois fluxos principais. Em seguida, a persistência: o modelo ER, o ORM e como garantimos que o saldo nunca fica errado. Aí vem o tutorial em cinco passos, a demonstração ao vivo e, por fim, os testes e a evolução dos modelos.

SLIDE 03 de 25

Reconhecimento do mérito em moedas

1:00 → 2:00

O objetivo do sistema é estimular o mérito estudantil. Cada professor recebe mil moedas por semestre. Quando um aluno participa bem da aula, o professor envia moedas para ele e é obrigado a escrever o motivo. O aluno recebe um email, acompanha tudo no extrato e, quando quiser, troca as moedas por uma vantagem: um desconto no restaurante, um material, um desconto na mensalidade. Na troca, ele recebe um cupom com código, que a empresa confere na hora.

São quatro perfis: aluno, professor, empresa parceira e administrador. Alunos e empresas se cadastram sozinhos. Professores e instituições já vêm pré-cadastrados, porque a instituição manda a lista no momento

da parceria. Os quinze casos de uso que modelamos na Sprint 1 estão todos implementados.

SLIDE 04 de 25

Requisitos e como foram atendidos

2:00 → 3:00

Aqui eu liguei cada requisito do enunciado ao que o atende no código. Login para todos os perfis: um token de sessão e um interceptor que confere o perfil antes de qualquer endpoint. No cadastro do aluno, as instituições vêm de uma lista, e o CPF é validado pelos dígitos e não pode repetir. As mil moedas por semestre são creditadas por um agendador, uma vez por semestre, somando ao que sobrou, ou seja, de forma acumulativa.

No envio, o motivo é obrigatório e o saldo é conferido com a conta bloqueada. O email sai depois que a operação foi gravada. E no resgate, o saldo é debitado, um código é gerado e saem dois emails, um para o aluno e outro para a empresa. O bloqueio do

saldo e o email depois do commit eu detalho mais à frente.

SLIDE 05 de 25

Tecnologias escolhidas

3:00 → 3:45

O back-end é Java 17 com Spring Boot 3.3, que é o padrão de mercado: já traz o servidor embutido, a injeção de dependências e integração pronta com banco, validação e email. A API é feita com Spring Web, em JSON, e os dados de entrada são validados com as anotações do Bean Validation. A persistência usa Spring Data JPA com Hibernate, então não escrevemos SQL à mão.

O banco padrão é o H2 em arquivo, para rodar sem instalar nada. Em produção, basta ativar o perfil do PostgreSQL. As senhas usam hash BCrypt, os emails usam o Spring Mail, e o front-end é HTML, CSS e JavaScript puro, para o foco ficar na arquitetura.

SLIDE 06 de 25

Diagrama de componentes (versão final)

3:45 → 5:00

Este é o diagrama de componentes na versão final, organizado de cima para baixo, no caminho de uma requisição. Em verde, a View: as páginas de cada perfil, que chamam a API levando o token. Em vermelho, a segurança, que intercepta tudo antes de chegar a um controller. Em laranja, os controllers, um por área. Em azul, os serviços, onde estão as regras de negócio. E em roxo, os repositórios, que acessam o banco pelo JPA.

À direita ficam o agendador do crédito semestral e o NotificacaoService, que recebe um evento depois de cada transação e cuida dos emails. Repare que nenhuma seta pula camada: o controller nunca acessa o banco direto. Isso é o MVC aplicado de verdade, e é o que deixa o modelo

alinhado com o código, que é um dos critérios da avaliação.

SLIDE 07 de 25

O caminho de uma requisição

5:00 → 5:45

Para ficar concreto, este é o caminho quando o aluno clica em resgatar. O JavaScript monta o JSON e manda o token. A segurança confere se o token é válido e se o perfil tem permissão. O controller valida o DTO com @Valid e chama o serviço. O serviço aplica as regras dentro de uma transação, e o repositório acessa o banco.

Dois detalhes. Os DTOs são records que definem o que entra e o que sai da API; por isso o hash da senha nunca aparece numa resposta. E todo erro sai no mesmo formato: 400 para validação, 401 sem login, 403 sem permissão, 404 para não encontrado e 422 para regra de negócio. O front-end usa esse formato para marcar o campo errado.

SLIDE 08 de 25

Autenticação e perfis de acesso

5:45 → 6:45

A autenticação tem quatro passos. No login, a senha é conferida com BCrypt e o sistema devolve um token aleatório de 256 bits. O front guarda esse token e o manda em toda requisição, no cabeçalho Authorization: Bearer. O interceptor bloqueia a requisição sem token, com 401, ou com perfil errado, com 403. E o aluno só vê e edita os próprios dados; o administrador vê todos.

No código, criamos duas anotações: @RequerPerfil, que restringe o endpoint, e @Publico, que libera, como no cadastro de aluno. O @UsuarioLogado injeta quem fez a requisição. Por isso o extrato não recebe id na URL: ele sempre usa o id de quem está logado, e ninguém consegue ver o extrato de outra

pessoa. Usamos um interceptor em vez do Spring Security completo para o mecanismo ficar visível e simples de explicar.

SLIDE 09 de 25

Envio de moedas pelo professor

6:45 → 7:30

Primeiro fluxo: o envio de moedas. O professor escolhe o aluno, a quantidade e escreve o motivo. A validação já barra motivo vazio ou valor menor ou igual a zero. No serviço, primeiro as contas do professor e do aluno são bloqueadas no banco. Depois vêm as regras: saldo suficiente e aluno da mesma instituição. Se alguma falhar, volta 422 com a mensagem.

Se estiver tudo certo, o serviço debita do professor, credita no aluno, grava a transferência e publica um evento. Quando a transação é confirmada, o NotificacaoService registra o email para o aluno, com o motivo e o novo saldo.

SLIDE 10 de 25

Resgate de vantagem com cupom

7:30 → 8:15

Segundo fluxo: o resgate de vantagem. O interceptor confere que quem pediu é um aluno, e no serviço a conta dele fica bloqueada até o fim. Se a vantagem estiver inativa ou o saldo não der, ele recebe o erro; é o bloco alt do diagrama. No caminho válido, o saldo é debitado e é gerado um código de doze caracteres, conferido para nunca repetir, e o resgate fica pendente.

Depois do commit, saem dois emails, para o aluno e para a empresa, com o mesmo código. Na troca presencial, a empresa digita o código e o cupom passa a utilizado. O mesmo cupom não pode ser usado duas vezes.

SLIDE 11 de 25

Modelo entidade-relacionamento

8:15 → 9:00

Entrando na persistência, este é o modelo ER: doze tabelas, todas geradas pelo Hibernate a partir das entidades. Temos duas hierarquias. Em cima, a tabela usuario, com os dados de acesso comuns, e uma tabela para cada perfil. Embaixo, a tabela transacao, com data e valor, e uma tabela para cada tipo de movimentação: transferência, resgate e crédito. O extrato é uma consulta sobre essa hierarquia.

A integridade também está no banco: CPF, CNPJ, email, login e cupom são únicos, e uma restrição de professor mais semestre impede creditar as mil moedas duas vezes. O professor tem chave estrangeira obrigatória para a instituição, como pede o enunciado.

SLIDE 12 de 25

Do diagrama de classes às tabelas (ORM)

9:00 → 9:45

Aqui está como o diagrama de classes virou banco, através das anotações JPA. As duas heranças usam JOINED: uma tabela por subclasse, ligada pela chave primária. O endereço é uma composição um para um, então é @Embeddable, e as colunas ficam na própria tabela aluno. O professor tem um @ManyToOne obrigatório, que vira uma FK NOT NULL. As vantagens têm cascata: excluir a empresa remove as vantagens dela. E o status do cupom é um enum gravado como texto.

Por que JOINED e não uma tabela única? Porque assim email e login ficam únicos entre todos os perfis e não sobram colunas nulas. O custo é um JOIN, tranquilo para o tamanho do sistema.

SLIDE 13 de 25

Repositórios: o padrão DAO sem SQL manual

9:45 → 10:30

Para acessar o banco usamos o padrão Repository, que é a versão do Spring para o DAO. A gente declara só a interface, estendendo JpaRepository, e o Spring gera a implementação, com save, findById e delete prontos.

O código mostra os três tipos de método que usamos. O existsByCpfAndIdNot é uma consulta gerada pelo nome do método, que confere se o CPF já existe em outro aluno. O @EntityGraph traz a instituição no mesmo SELECT e evita o problema de N+1 consultas na listagem. E o @Lock PESSIMISTIC_WRITE faz um SELECT FOR UPDATE, que bloqueia a linha enquanto o saldo muda. São onze repositórios, um por entidade.

SLIDE 14 de 25

Nenhuma moeda some ou aparece do nada

10:30 → 11:15

A persistência se resume a uma frase: nenhuma moeda some ou aparece do nada. Isso vem de três garantias. A primeira é a atomicidade: com @Transactional, debitar, creditar e gravar é uma operação só; se uma parte falha, tudo é desfeito. A segunda é a concorrência: com o bloqueio pessimista, se dois resgates chegam ao mesmo tempo, o segundo espera o primeiro e já vê o saldo atualizado, então não dá para gastar o mesmo saldo duas vezes.

A terceira é a consistência: o email usa um listener na fase AFTER_COMMIT, então só é registrado se a operação foi gravada. Uma transação desfeita nunca avisa o aluno de moedas que ele não recebeu.

SLIDE 15 de 25

Do zero a um endpoint com Spring Boot

11:15 → 11:30

Agora o tutorial. A ideia é que qualquer um aqui consiga reproduzir essa stack num projeto próprio. Vou mostrar em cinco passos como construímos o cadastro de vantagens, e o mesmo padrão se repete em todo o sistema.

SLIDE 16 de 25

Criar o projeto no Spring Initializr

11:30 → 12:15

Passo um: criar o projeto. No site start.spring.io, escolhemos Maven, Java 17 e Spring Boot 3.3 e marcamos as dependências: Spring Web, Spring Data JPA, Validation, os drivers do H2 e do PostgreSQL e o Java Mail Sender. O projeto já vem com o Maven Wrapper, então quem clonar só precisa ter o Java instalado.

Depois organizamos os pacotes pelas camadas, como está à direita: controller, dto, service, model e repository. Essa estrutura espelha o diagrama de componentes. No application.properties bastam duas linhas: o banco num arquivo H2 e o Hibernate criando as tabelas sozinho.

SLIDE 17 de 25

Mapear a entidade e criar o repositório

12:15 → 13:00

Passo dois: a entidade e o repositório. A classe vira tabela com @Entity. O @Id com @GeneratedValue é a chave primária com auto incremento, o @Column define não nulo e tamanho, e o @ManyToOne com @JoinColumn vira a chave estrangeira para a empresa. Nenhum CREATE TABLE foi escrito à mão.

O repositório é só uma interface e já tem todo o CRUD. O método findByAtivaTrueOrderByCustoAsc vira a consulta das vantagens ativas ordenadas pelo custo, que é exatamente o catálogo que o aluno vê.

SLIDE 18 de 25

Regras no serviço, HTTP no controller

13:00 → 13:45

Passo três: a regra fica no serviço e o HTTP no controller. O serviço tem @Service e @Transactional e recebe o repositório pelo construtor: o Spring injeta, a gente nunca dá new. O método cadastrar monta a vantagem, liga à empresa e salva.

O controller só traduz HTTP: o @PostMapping define a rota, o @PathVariable pega o id da empresa e o @RequestBody converte o JSON no record. O ponto principal é o @Valid, que aplica as anotações do record, como @NotBlank e @Positive. Se o dado estiver errado, o Spring devolve 400 antes de chegar ao serviço. No sucesso, a resposta é 201 Created.

SLIDE 19 de 25

O front-end conversa com a API

13:45 → 14:15

Passo quatro: o front-end. Toda a comunicação fica numa função do api.js: ela faz o fetch, coloca o token no cabeçalho e, se der erro, lança o JSON com a mensagem e os campos. Então, em qualquer formulário, o código é o mesmo: tenta enviar e, se falhar, o mostrarErros marca cada campo com problema. As páginas ficam na pasta static e o próprio Spring serve tudo na mesma porta. Se a sessão expirar, o front volta para o login.

SLIDE 20 de 25

Rodar e testar

14:15 → 14:45

Último passo: rodar e testar. Com um comando, ./mvnw spring-boot:run, o sistema abre em localhost:8080 já com dados de exemplo; a senha de todos é 123456. Para testar, ./mvnw test. Os testes de integração sobem a aplicação com banco em memória e chamam a API com o MockMvc, do mesmo jeito que o front faz, com token e tudo. Esse teste à direita percorre o fluxo inteiro: envio, extrato, resgate e conferência do cupom.

SLIDE 21 de 25

Saldo, vantagens e cupom na mesma tela

14:45 → 16:15] DEMONSTRAÇÃO AO VIVO

(Antes da aula: apagar a pasta data/ e subir o sistema, para começar com os dados zerados.)

Agora vou mostrar o sistema funcionando.

[Abrir localhost:8080 e clicar em carla.mendes]

Estou logado como a professora Carla, que tem mil moedas. Vou escolher a Ana Souza, colocar 300 moedas e o motivo "Excelente apresentação do trabalho".

[Clicar em Enviar moedas]

O saldo caiu para 700, e o envio já aparece nos últimos envios.

[Clicar em Sair e entrar como ana.souza]

Agora sou a Ana. O saldo está em 300 e, no extrato, aparece o recebimento com o nome da professora e o motivo. Na aba Emails está o aviso que o sistema mandou.

[Voltar para a aba Vantagens]

O botão Resgatar só aparece quando o saldo cobre o custo. Vou resgatar o almoço, de 150 moedas.

[Confirmar o resgate]

Aqui está o código do cupom, que também foi enviado por email. Vou anotar esse código.

(Se a rede ou o sistema falharem, apresente as duas telas deste slide.)

SLIDE 22 de 25

Do reconhecimento à troca presencial

16:15 → 17:15] CONTINUAÇÃO DA DEMONSTRAÇÃO

[Clicar em Sair e entrar como cantina]

Agora sou a empresa Cantina do Campus. Na aba Resgates e cupons, o resgate da Ana aparece como aguardando troca. Na troca presencial, eu digito o código que ela mostrou...

[Digitar o código e clicar em Conferir]

...e o cupom passa a utilizado. Se eu tentar conferir de novo, o sistema recusa, porque o cupom já foi usado.

[Opcional, se sobrar tempo: entrar como admin]

O administrador tem o CRUD completo de alunos e empresas, cadastra e importa professores e vê todos os emails que o sistema enviou.

(Se a demonstração ao vivo não funcionar, use as duas telas deste slide.)

SLIDE 23 de 25

Testes automatizados

17:15 → 17:45

(Antes da aula: rodar ./mvnw test e confirmar que todos passam.)

Sobre a qualidade: são dez cenários de teste automatizado, em três classes. Testamos os CRUDs, inclusive CPF e CNPJ duplicados; as regras de acesso, com 401 sem login e 403 para perfil errado ou dados de outro usuário; as regras de moeda, como motivo obrigatório, saldo insuficiente e aluno de outra instituição; e o cupom, com o código nos dois emails, a conferência única e o crédito semestral sem duplicar. Tudo roda com um comando: ./mvnw test.

SLIDE 24 de 25

Os modelos evoluíram junto com o código

17:45 → 18:15

Os modelos evoluíram junto com o código. Na Sprint 1 fizemos a modelagem: casos de uso, histórias, diagrama de classes e de componentes. Na Sprint 2 veio a persistência, com o modelo ER, os repositórios e os primeiros CRUDs, e atualizamos os diagramas de classes e de componentes. Na Sprint 3, com o sistema completo, entraram o perfil de acesso, a conferência do cupom e o registro de emails, e criamos os diagramas de sequência. Todas as versões estão no repositório, em PlantUML, versionadas junto com o código.

SLIDE 25 de 25

Obrigado!

18:15 → 18:30, depois perguntas

É isso. Obrigado pela atenção! O link do repositório e as contas de teste estão na tela, para quem quiser rodar. Fico à disposição para perguntas.

RESPOSTAS RÁPIDAS, se perguntarem:

• Por que interceptor e não Spring Security? Para o mecanismo ficar simples e visível. O projeto só precisava de token e perfil, e o interceptor resolve isso em poucas classes.

• Por que JOINED? Email e login ficam únicos entre os perfis e não sobram colunas nulas. O custo é só um JOIN.

• Como o crédito semestral não duplica? Há uma restrição única de professor mais semestre no banco, e o serviço confere antes de creditar. Rodar duas vezes não muda nada.

• E se dois resgates chegarem juntos? O SELECT FOR UPDATE faz o segundo esperar o primeiro, então o saldo nunca fica negativo.

• O email é enviado de verdade? Sim, se configurar um SMTP no application.properties. Sem isso, fica em modo simulado: registrado no banco e visível na aba Emails.

• Onde fica a regra de negócio? Sempre nos serviços. O controller só recebe e valida, e o repositório só acessa o banco.